МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ (НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Кафедра 806 «Вычислительная математика и программирование»

Отчет по лабораторной работе №2 по курсу «Операционные системы»

Тема: «Управление процессами и межпроцессное взаимодействие посредством каналов (Pipe)»

Выполнил(а): А. Е. Пронина

Группа: М8О-203БВ-25

Преподаватель: Е. С. Миронов

Москва, 2026

Условие

Цель работы: приобретение практических навыков управления процессами в операционной системе и обеспечения обмена данными между ними посредством неименованных каналов (pipe), а также перенаправления стандартных потоков ввода-вывода дочерних процессов.

Задание: родительский процесс создает два дочерних процесса. Первой и второй строкой пользователь в консоль родительского процесса вводит имена файлов, которые будут использованы для открытия на запись для child1 и child2 соответственно. Родительский и дочерний процессы представлены разными программами. Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1 или в pipe2 в зависимости от правила фильтрации. Процессы child1 и child2 производят работу над строками и пишут результаты своей работы в стандартный вывод (перенаправленный в файлы).

Вариант: 18 (Группа 5). Правило фильтрации: нечетные строки отправляются в pipe1, четные в pipe2. Дочерние процессы удаляют все гласные из строк.

Метод решения

Программа реализована с использованием кроссплатформенного подхода, абстрагирующего системные вызовы POSIX (Linux/macOS) и Win32 API (Windows).

Алгоритм работы родительского процесса:

1. Запрос имен двух выходных файлов у пользователя и их открытие на запись с флагами создания/усечения (`O_WRONLY | O_CREAT | O_TRUNC` / `CREATE_ALWAYS`).
2. Создание двух неименованных каналов (`pipe` / `CreatePipe`).
3. Двукратный вызов функции создания дочернего процесса. Перед запуском исполняемого файла дочернего процесса стандартный поток ввода (`stdin`) перенаправляется на читающий конец соответствующего канала, а стандартный поток вывода (`stdout`) — на дескриптор открытого файла. В POSIX это достигается через `dup2`, в Windows — через структуру `STARTUPINFO` с флагом `STARTF_USESTDHANDLES`.
4. Родительский процесс закрывает все ненужные ему концы каналов (читающие концы обоих каналов), чтобы избежать утечек дескрипторов.
5. В цикле считываются строки из `stdin`. Нечетные строки записываются в `pipe1`, четные — в `pipe2`.
6. После получения признака конца файла (EOF) от пользователя, родительский процесс закрывает пишущие концы каналов. Это критически важно: только когда все дескрипторы пишущего конца закрыты, системный вызов `read` в дочернем процессе вернет 0 (EOF), позволяя ему корректно завершить цикл чтения.
7. Ожидание завершения дочерних процессов (`waitpid` / `WaitForSingleObject`) и вывод их кодов возврата.

Описание программы

Проект разделен на модули для обеспечения чистоты архитектуры и соответствия инженерным требованиям:

- `headers/*.h`: объявление кроссплатформенных структур и прототипов функций.
- `realization/*_posix.c` и `realization/*_win.c`: реализация системно-зависимых функций (`open_file_for_write`, `create_pipe`, `create_child_process`).
- `parent.c`: основная логика оркестрации процессов.
- `child.c`: логика обработки данных (удаление гласных).

Ключевой фрагмент перенаправления потоков в дочернем процессе (POSIX):

```
1 if (pid == 0) { //
2     if (stdin_handle != -1) {
3         if (dup2((int)stdin_handle, STDIN_FILENO) == -1) {
4             _exit(1);
5         }
6     }
7     if (stdout_handle != -1) {
8         if (dup2((int)stdout_handle, STDOUT_FILENO) == -1) {
```

```
9               _exit(1);
10          }
11      }
12      execvp(executable, argv);
13      _exit(1);
14  }
```

Результаты работы

Программа была протестирована на наборе из шести строк, содержащих как гласные, так и согласные буквы. Ввод строк завершался нажатием Ctrl+D (сигнал EOF). Ниже приведены примеры работы системы.

Консольный вывод родительского процесса

В консоли отображаются только сообщения родительского процесса: его PID, запрос имен файлов, PID созданных дочерних процессов и коды их завершения после окончания работы.

Листинг 1: Вывод в консоль родительского процесса

```
$ ./parent
[Parent] PID: 89233
Enter output file for Child1: ch1
Enter output file for Child2: ch2
[Parent] Child1 created with PID: 89265
[Parent] Child2 created with PID: 89266
Enter lines (Ctrl+D / Ctrl+Z to finish):
fdg
sdfon
ainaaon
aaass
aiaoaiau
jnv
[Parent] Child1 (PID: 89265) exit code: 0
[Parent] Child2 (PID: 89266) exit code: 0
[Parent] Results written to: ch1, ch2
```

Содержимое выходных файлов

Поскольку стандартный поток вывода (`stdout`) дочерних процессов перенаправлен в файлы, служебные сообщения о запуске и завершении, а также результаты обработки строк, записываются непосредственно в файлы.

Файл `ch1` содержит результаты обработки **нечетных** строк (1, 3, 5):

- Строка 1: `fdg` → `fdg` (гласных нет)
- Строка 3: `ainaaon` → `nn` (удалены a, i, a, a, o)
- Строка 5: `aiaoaiau` → (все символы — гласные, остается только символ перевода строки)

Листинг 2: Содержимое файла ch1

```
[Child1] PID: 89265, started
fdg
nn

[Child1] PID: 89265, finished
```

Файл `ch2` содержит результаты обработки **четных** строк (2, 4, 6):

- Строка 2: `sdfon` → `sdfn` (удалена o)
- Строка 4: `aaass` → `ss` (удалены a, a, a)
- Строка 6: `jnv` → `jnv` (гласных нет)

Листинг 3: Содержимое файла ch2

```
[Child2] PID: 89266, started
sdfn
ss
jnv
[Child2] PID: 89266, finished
```

Анализ результатов

Результаты полностью соответствуют заданию Варианта 18:

1. Нечетные строки корректно перенаправлены в `pipe1` и обработаны дочерним процессом 1, результат записан в файл `ch1`.
2. Четные строки корректно перенаправлены в `pipe2` и обработаны дочерним процессом 2, результат записан в файл `ch2`.
3. Все гласные буквы (a, e, i, o, u) удалены из строк.
4. Служебные сообщения дочерних процессов (PID, статус запуска/завершения) присутствуют в выходных файлах, что подтверждает корректность перенаправления потока `stdout` в файлы посредством системных вызовов `dup2` (POSIX) / `STARTUPINFO` (Win32).
5. Оба дочерних процесса завершились с кодом возврата 0, что свидетельствует об отсутствии ошибок выполнения и корректном получении EOF после закрытия родительским процессом пишущих концов каналов.

Диагностика системных вызовов (strace)

Для подтверждения использования нативных системных примитивов ОС был проведен анализ профиля системных вызовов программы с помощью утилиты `strace`. Запуск осуществлялся с флагом `-f` для трассировки всех дочерних процессов и фильтрацией по категориям `process, ipc, write`.

Ниже приведены ключевые фрагменты реального лога выполнения, демонстрирующие жизненный цикл процессов и межпроцессное взаимодействие:

Листинг 4: Фрагмент вывода strace (ключевые системные вызовы)

```
strace -f -e trace=process,ipc,write ./parent
execve("./parent", ["./parent"], 0x7ffdfdb7dcc8 /* 38 vars */) = 0
write(1, "[Parent] PID: 91777\n", 20[Parent] PID: 91777
) = 20
write(1, "Enter output file for Child1: ", 30Enter output file for Child1: ) = 30
child1
write(1, "Enter output file for Child2: ", 30Enter output file for Child2: ) = 30
child2
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLDstrace:
    Process 91850 attached
, child_tidptr=0x7f572d39fa10) = 91850
[pid 91777] clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|
    SIGCHLDstrace: Process 91851 attached
 <unfinished ...>
[pid 91850] execve("./child", ["./child", "child", "1"], 0x7ffddee99fe8 /* 38 vars */
    <unfinished ...>
[pid 91777] <... clone resumed>, child_tidptr=0x7f572d39fa10) = 91851
[pid 91777] write(1, "[Parent] Child1 created with PID"..., 40[Parent] Child1 created
    with PID: 91850
) = 40
[pid 91777] write(1, "[Parent] Child2 created with PID"..., 40[Parent] Child2 created
    with PID: 91851
) = 40
[pid 91851] execve("./child", ["./child", "child", "2"], 0x7ffddee99fe8 /* 38 vars */
    <unfinished ...>
[pid 91777] write(1, "Enter lines (Ctrl+D / Ctrl+Z to "..., 41Enter lines (Ctrl+D /
    Ctrl+Z to finish):
) = 41
[pid 91851] write(1, "[Child2] PID: 91851, started\n", 29 <unfinished ...>
[pid 91850] write(1, "[Child1] PID: 91850, started\n", 29 <unfinished ...>

aaaa
[pid 91777] write(4, "aaaa\n", 5) = 5
[pid 91850] write(1, "\n", 1) = 1

ooooo
[pid 91777] write(6, "ooooo\n", 6) = 6
[pid 91851] write(1, "\n", 1) = 1


lalalal
[pid 91777] write(4, "lalalal\n", 8) = 8
[pid 91850] write(1, "llll\n", 5) = 5
aaiaaoa
[pid 91777] write(6, "aaiaaoa\n", 8) = 8
[pid 91851] write(1, "\n", 1) = 1
[pid 91777] wait4(91850 <unfinished ...>
```

```
[pid 91850] write(1, "[Child1] PID: 91850, finished\n", 30 <unfinished ...>
[pid 91851] write(1, "[Child2] PID: 91851, finished\n", 30) = 30
[pid 91851] exit_group(0 <unfinished ...>
[pid 91850] exit_group(0 <unfinished ...>
[pid 91851] <... exit_group resumed>) = ?
[pid 91850] <... exit_group resumed>) = ?
[pid 91850] +++ exited with 0 +++
[pid 91777] <... wait4 resumed>, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) =
    91850
[pid 91851] +++ exited with 0 +++
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=91850, si_uid=1000,
    si_status=0, si_utime=0, si_stime=0} ---
wait4(91851, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 91851
write(1, "[Parent] Child1 (PID: 91850) exi"..., 42[Parent] Child1 (PID: 91850) exit
    code: 0
) = 42
write(1, "[Parent] Child2 (PID: 91851) exi"..., 42[Parent] Child2 (PID: 91851) exit
    code: 0
) = 42
write(1, "[Parent] Results written to: chi"..., 44[Parent] Results written to: child1,
     child2
) = 44
exit_group(0) = ?
+++ exited with 0 +++
```

Данный лог наглядно подтверждает, что программа использует требуемые низкоуровневые системные вызовы ядра Linux (`clone`, `execve`, `write`, `wait4`) для управления процессами и организации IPC, без применения высокоуровневых библиотечных оберток.

Выводы

В ходе выполнения лабораторной работы была спроектирована и реализована кроссплатформенная система межпроцессного взаимодействия (IPC) с использованием неименованных каналов. Практическая реализация позволила глубже изучить механизмы, лежащие в основе управления процессами и обмена данными в операционных системах.

Ключевым теоретическим аспектом, подтвержденным на практике, является природа каналов (`pipe`) как однонаправленных потоков байтов, буферизуемых в адресном пространстве ядра. Системный вызов `pipe()` (POSIX) или `CreatePipe()` (Win32) создает пару файловых дескрипторов: один для чтения, другой для записи. Данные, записанные в один конец канала, могут быть прочитаны из другого в порядке FIFO. Важной особенностью является то, что системный вызов `read()` блокирует процесс-читатель до тех пор, пока не появятся данные или пока все дескрипторы пишущего конца не будут закрыты. Именно поэтому критически важно, чтобы родительский процесс закрыл *все* копии дескрипторов, связанных с пишущим концом канала после `fork()`, иначе дочерний процесс будет бесконечно блокироваться в ожидании данных, что приведет к взаимоблокировке (deadlock).

Механизм создания дочерних процессов в POSIX реализуется парой вызовов `fork()` и `exec()`. Вызов `fork()` создает точную копию текущего процесса, наследующую все открытые файловые дескрипторы, что является основой для последующего перенаправления потоков. Вызов `execvp()` заменяет образ памяти дочернего процесса на исполняемый файл, сохраняя при этом перенаправленные дескрипторы. В Windows аналогичная функциональность инкапсулирована в едином системном вызове `CreateProcess()`, который принимает структуру `STARTUPINFO` с флагом `STARTF_USESTDHANDLES` для явного

указания дескрипторов стандартных потоков.

Перенаправление стандартных потоков ввода-вывода достигается посредством системного вызова `dup2()` (POSIX), который дублирует файловый дескриптор, заменяя им стандартные дескрипторы `STDIN_FILENO` (0) и `STDOUT_FILENO` (1). Этот механизм демонстрирует принцип прозрачности файловых дескрипторов в UNIX-подобных системах: дочерняя программа (`child.c`) работает исключительно с абстракциями `stdin` и `stdout`, не содержа в себе никакой логики работы с каналами или файлами. Операционная система сама маршрутизирует данные между процессами на уровне ядра.

Ожидание завершения дочерних процессов реализовано через `waitpid()` (POSIX) и `WaitForSingleObject()` (Win32). Эти вызовы не только блокируют родительский процесс до завершения дочернего, но и позволяют получить код возврата, а также освободить запись в таблице процессов ядра. Без вызова `wait()` завершенный дочерний процесс остается в состоянии "зомби"(zombie), занимая место в таблице процессов до завершения родителя.

Также была отработана инженерная практика безопасного управления ресурсами: каждый системный вызов (`fork`, `pipe`, `open`) сопровождается проверкой кода возврата, а выделенные дескрипторы гарантированно освобождаются при любом сценарии выполнения, включая аварийное завершение. Это предотвращает утечки ресурсов и появление зомби-процессов, что соответствует требованиям к надежному системному программированию.

Список литературы

1. Таненбаум Э., Бос Х. Современные операционные системы. – 4-е изд. – СПб.: Питер, 2015.